

```

import pandas as pd
import re

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# =====
# DATASET
# =====

data = {
    "text": [
        "India defeated Australia in the cricket final",
        "Virat Kohli scored a century in the ODI match",
        "The football team won the championship",
        "The tennis player reached the final round",
        "The basketball tournament attracted many fans",

        "Stock markets closed higher after strong earnings",
        "The company announced a new investment plan",
        "Investors are optimistic about economic growth",
        "The bank reported increased quarterly profits",
        "Oil prices affected global trade markets",

        "Scientists discovered a new planet in space",
        "Researchers developed an advanced AI system",
        "A new vaccine showed promising results",
        "Technology companies are investing in robotics",
        "The space mission successfully reached orbit",

        "World leaders met to discuss climate change",
        "The government signed an international agreement",
        "A peace treaty was announced between nations",
        "The election results were declared yesterday",
        "Global organizations discussed economic policies"
    ],
    "label": [
        "Sports", "Sports", "Sports", "Sports", "Sports",
        "Business", "Business", "Business", "Business", "Business",
        "Science", "Science", "Science", "Science", "Science",
        "World", "World", "World", "World", "World"
    ]
}

df = pd.DataFrame(data)

print("Dataset Loaded Successfully")
print(df.head())

# =====
# TEXT PREPROCESSING
# =====

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z ', '', text)
    return text

df["clean_text"] = df["text"].apply(clean_text)

# =====

```

```

# TF-IDF
# =====

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(df["clean_text"])

# =====
# UNSUPERVISED LEARNING
# K-MEANS
# =====

kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

df["Cluster"] = kmeans.fit_predict(X)

print("\nCluster Distribution")
print(df["Cluster"].value_counts())

# =====
# SUPERVISED LEARNING
# =====

label_map = {
    "Sports": 0,
    "Business": 1,
    "Science": 2,
    "World": 3
}

df["target"] = df["label"].map(label_map)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    df["target"],
    test_size=0.2,
    random_state=42
)

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# PREDICTION
# =====

sample_news = [
    "India won the cricket world cup"
]

sample_clean = [clean_text(x) for x in sample_news]

sample_vector = vectorizer.transform(sample_clean)

prediction = model.predict(sample_vector)

```

```
reverse_map = {
    0: "Sports",
    1: "Business",
    2: "Science",
    3: "World"
}

print("\nNews:", sample_news[0])
print("Predicted Category:", reverse_map[prediction[0]])
```

Dataset Loaded Successfully

	text	label
0	India defeated Australia in the cricket final	Sports
1	Virat Kohli scored a century in the ODI match	Sports
2	The football team won the championship	Sports
3	The tennis player reached the final round	Sports
4	The basketball tournament attracted many fans	Sports

Cluster Distribution

Cluster

2	8
0	6
1	4
3	2

Name: count, dtype: int64

Accuracy:

0.0

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2.0
1	0.00	0.00	0.00	0.0
2	0.00	0.00	0.00	0.0
3	0.00	0.00	0.00	2.0
accuracy			0.00	4.0
macro avg	0.00	0.00	0.00	4.0
weighted avg	0.00	0.00	0.00	4.0

News: India won the cricket world cup

Predicted Category: Business

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision-Recall scores were not calculated because the predicted labels do not match the true labels.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision-Recall scores were not calculated because the predicted labels do not match the true labels.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision-Recall scores were not calculated because the predicted labels do not match the true labels.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision-Recall scores were not calculated because the predicted labels do not match the true labels.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision-Recall scores were not calculated because the predicted labels do not match the true labels.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision-Recall scores were not calculated because the predicted labels do not match the true labels.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

